

DESIGN & CALCULATIONS**Truth Table:**

States: (6 total)

State	Binary	Output [entrance_gate: exit_gate]	Definition
S0	000	00	Idle~ no cars at entrance, watching for cars at entrance
S1	001	00	Wait_in~ car detected at entrance → counter runs to 5, then password checked
S2	010	10	Open_in~ correct entrance pw detected → entrance gate
S3	011	00	Parked~ car inside parking structure, watching for car at exit gate
S4	100	00	Wait_out~ car detected at exit → counter runs to 5, then password checked
S5	101	01	Open_out~ correct exit password detected → exit gate open

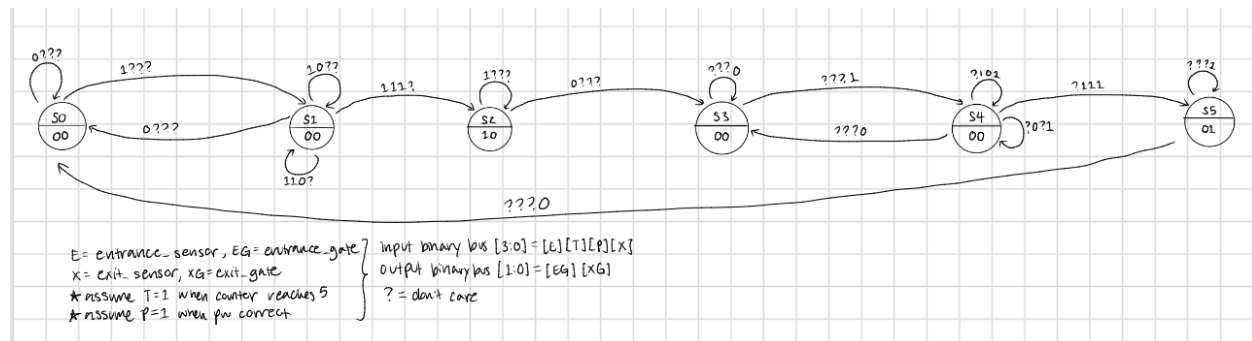
Current/ Next-State:

- entrance_sensor(E), count==5(T), pw_ok(P), exit_sensor(X)
- Entrance_gate(EG), exit_gate(XG)

Present State	Bin Code Q[2:0]	Input bus ETPX[3:0]	Next State	Bin Code N[2:0]	Output bus EgXg[1:0]
S0	000	0XXX	S0	000	00
S0	000	1XXX	S1	001	00
S1	001	0XXX	S0	000	00
S1	001	10XX	S1	001	00

S1	001	110X	S1	001	00
S1	001	111X	S2	010	00
S2	010	1XXX	S2	010	10
S2	010	0XXX	S3	011	10
S3	011	XXX0	S3	011	00
S3	011	XXX1	S4	100	00
S4	100	XXX0	S3	011	00
S4	100	X0X1	S4	100	00
S4	100	X101	S4	100	00
S4	100	X111	S5	101	00
S5	101	XXX1	S5	101	01
S5	101	XXX0	S0	000	01

State Diagram:



Logic Equations: (Moore type, outputs depend on curr state)

- entrance_sensor(E), count==5(T), pw_ok(P), exit_sensor(X)
- Entrance_gate(EG), exit_gate(XG)
- N[2:0] is next state bus, Q[2:0] is curr state bus forming SX states
 - $S0 = \sim Q[2] \& \sim Q[1] \& \sim Q[0]$
 - $S1 = \sim Q[2] \& \sim Q[1] \& Q[0]$
 - $S2 = \sim Q[2] \& Q[1] \& \sim Q[0]$
 - $S3 = \sim Q[2] \& Q[1] \& Q[0]$

- $S4 = Q[2] \& \sim Q[1] \& \sim Q[0]$
- $S5 = Q[2] \& \sim Q[1] \& Q[0]$

N[2]: when next state is S4, S5

- $N[2] = (S3) \& X \mid (S4) \& X \mid (S5) \& X$

N[1]: when next state is S2, S3

- $N[1] = (S1) \& ETP \mid (S2) \& \sim E \mid (S3) \& \sim X \mid (S4) \& \sim X$

N[0]: when next state is S1, S3, S5

- $N[0] = (S0) \& E \mid (S1) \& E \& (\sim T \mid \sim P) \mid (S2) \& \sim E \mid (S3) \& \sim X \mid (S4) \& (\sim X \mid XTP) \& (S5) \& X$

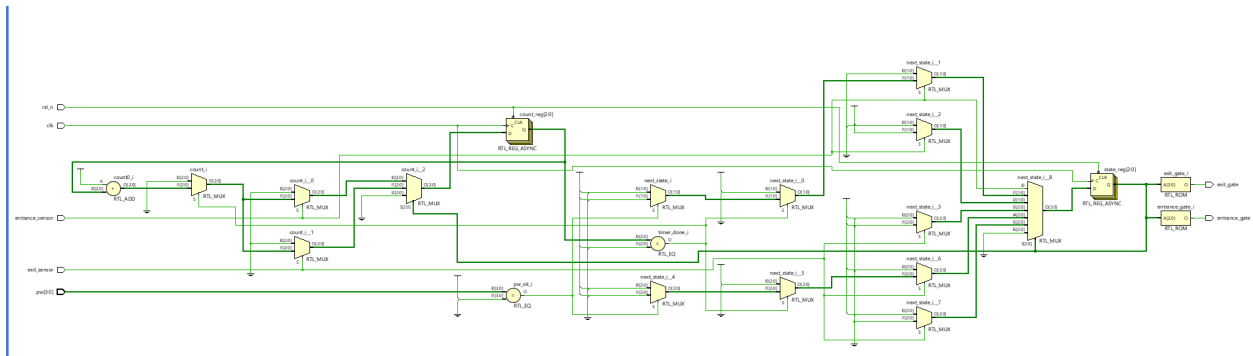
Assumptions:

- Entrance_sensor = 1 when car at entrance
- Exit_sensor = 1 when car at exit
- Assuming entered password pw = 4'bxxxx, with stored pw being fixed default val
- 2 gate outputs: entrance_gate, exit_gate
- Wait requirement implemented with counter [0, 5]
- Asynchronous active-low reset: rst_n

Inputs: clk, rst_n (**T**), entrance_sensor (**E**), exit_sensor (**X**), pw[3:0] (**P**)

Outputs: entrance_gate (**EG**), exit_gate (**XG**) (each signals gate to open)

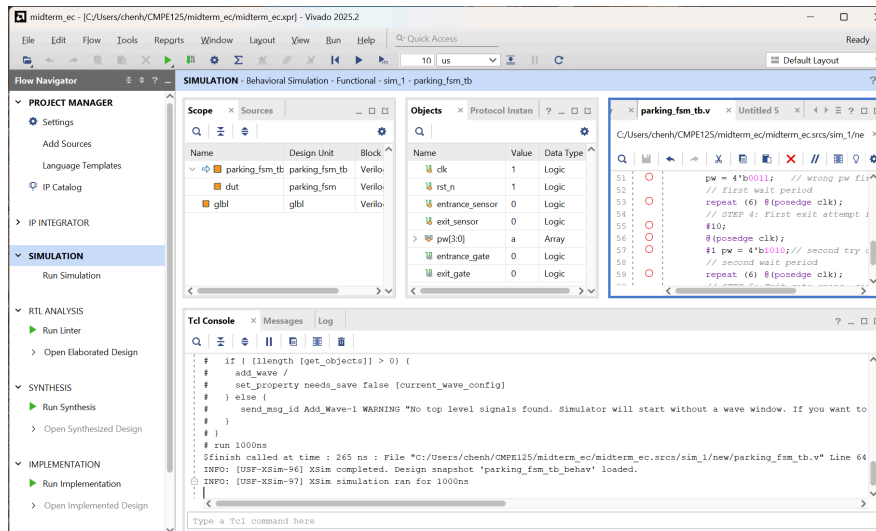
Circuit/ Block Diagrams:



Vivado-generated schematic

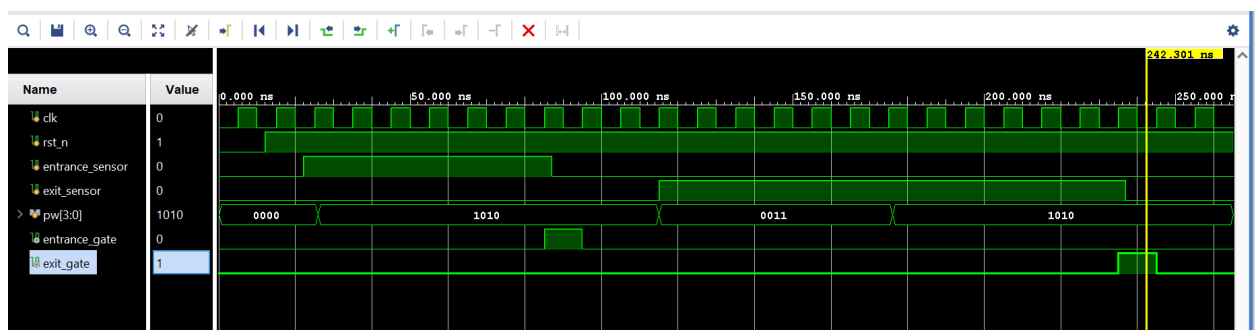
SCREENSHOTS

Compilation:

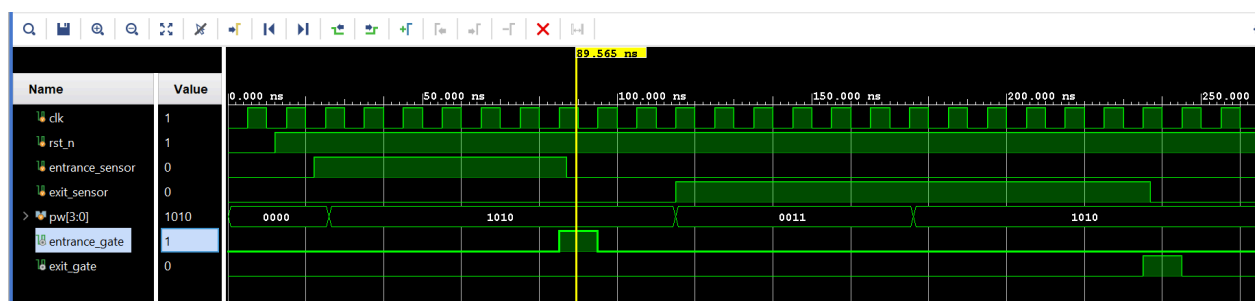


Console messages after simulation ran

Simulation Waveforms:



Exit_gate set when exit gate opens at end of simulation 5 clock cycles after correct password



Entrance_gate set when entrance opens after password matches + 5 clock cycles

SUMMARY

The objective of this problem is to model a car parking lot system as a Moore FSM with 6 states and an active-low reset toggle `rst_n`, which sends the FSM to S0 and resets all inputs. The progression of a car through the parking lot in the tested scenario should go as follows:

1. Car enters → `entrance_sensor` = 1, FSM goes from S0 to S1 on next clock, counter starts counting from 0 to 5
2. Password checked at entrance → count reaches 5, `pw_ok` = 1, FSM goes from S2 to S3, `entrance_gate` = 1
3. Car passes entrance gate → `entrance_sensor` = 0 → FSM goes from S2 to S3, `entrance_gate` = 0 again
4. Car moves to exit → `exit_sensor` = 1, FSM goes from S3 to S4, counter starts again
5. First exit try fails → count reaches 5, wrong pw applied, FSM stays in S4, `exit_gate` still 0
6. Second exit try succeeds → pw entered correctly, *after* next 5-count delay → `pw_ok`, FSM goes from S4 to S5, `exit_gate` = 1
7. Car leaves exit → `exit_sensor` = 0, FSM goes from S5 to S0, `exit_gate` = 0

When simulating the waveform, we can see at the beginning that `rst_n` = 0 and then goes to 1, forcing the FSM into an initial idle state while the reset is active. The `entrance_sensor` then goes high as pw changes from 0000 to 1010, meaning a car is detected and the correct password has been entered. While the `entrance_sensor` stays high, the FSM should leave idle, enter the `wait_in` stage, count until the timer finishes, and check the password before moving to the `open_in` stage. Hence, we see the `entrance_gate` in the waveform stay low for a while, then pulse high briefly around the time the waiting period finishes. Similarly, when the exit sequence begins, the `exit_sensor` goes high, the FSM should leave the parked state, enter the `wait_out` state, time the count, and check the password before moving on. In the waveform, we do not see `exit_gate` go high during the interval `p=0011` because it does not satisfy the condition for a correct password. Only when the correct password is entered while the `exit_sensor` is high does the FSM enter `open_out`. Finally, at the very end of the waveform, `exit_sensor` goes low and the FSM returns to idle as the car leaves;.

VERILOG CODE

See files attached for `parking_fsm.v` and `parking_fsm_tb.v`,

TROUBLESHOOTING

I think during the exam, I got caught off-guard by the addition of the password and I wasn't sure how to implement it. After reading the prompt, I spent a significant amount of time wondering how to give each driver the ability to set their own password, and how I was to implement memory into my FSM. Then, I realized I was overcomplicating things, and when I finally managed to think through the states, I ran out of time to derive the equations from the state table

and write the verilog code. Next time, I will just state my assumptions and try to move on with the most basic interpretation of the prompt.

During the development of this FSM at home, there were several issues I ran into during the implementation of the testbench. Despite the module logic appearing correct, on the first waveform, entrance_gate and exit_gate remained 0 throughout the simulation, indicating a problem with the timing of the testbench inputs. It turns out that the sensor signals were being deasserted too early before the FSM had enough clock cycles to enter the wait state, complete counting, and transition to the gate-open state. This was fixed by making the simulator to wait until the next rising edge of clk, and then wait 1 time unit after that before assigning pw, ensuring that pw changes slightly after the clk edge and avoids race conditions. We have to keep in mind that if we change a signal at exactly the same simulation time as the clock edge, it can lead to race conditions as testbench tries to change the pw at the same time that DUT samples it.